

第二章：数据的导入与导出

数据导入和导出是pandas中很基础且重要的一个部分。pandas提供了很多不同格式数据的导入和导出方法，可以将其他格式数据转为DataFrame格式。我们可以将list、dict格式数据转为dataFrame格式，也可以从本地的csv、json等文本格式数据和sql、MongoDB等数据库中读取和保存数据等等。下面就分别以三大类介绍一些常见的数据格式的导入与导出。

引言：

当我们开始着手做一个数据分析项目时，选择和导入数据集是第一个步骤，而导出数据虽然非必需，但有时候我们也需要保存处理或者分析后的结果，方便下次使用。在pandas中，它已经为我们提供了很多不同格式数据的导入和导出方法，本节将具体介绍一些较为常用的方法，包括excel、csv文件以及数据库的导入导出等。

1. list、dict、np.array 格式数据

1.1 list

一般读取一个list，生成的结果如下：

```
1 pd.DataFrame([1,2,3,4])
```

运行结果：

```
    0
0   1
1   2
2   3
3   4
```

如果读取的list中的每个元素都是一个元组，会发生什么呢?

```
pd.DataFrame([(1,2,3,4),(2,3,4,5)],columns = ['value1','value2','value3','value4'])
```

运行结果：

```
    value1  value2  value3  value4
0      1       2       3       4
1      2       3       4       5
```

如果忽略columns的话，第二个list的值不是列名，而是默认生成索引名，如下：

```
pd.DataFrame([(1,2,3,4),(2,3,4,5)],['value1','value2'])
```

运行结果：

```
         0   1   2   3
value1   1   2   3   4
value2   2   3   4   5
```

1.2 dict

这里我们以一个字典为数据，看下不同操作的结果有何不同。

```
data = {'a':[1,2],'b':[2,3]}
```

直接调用DataFrame进行读取的话，生成的DataFrame结构如下:

```
pd.DataFrame(data)
# 或者
pd.DataFrame.from_dict(data)
```

运行结果：

```
    a   b
0   1   2
1   2   3
```

需要说明的是：from_dict这个方法只有在pandas 0.23版本后才有，如果在早期的版本如0.19中调用会出现报错。
如果我们想以a，b作为索引，以list中的每个值分别为一列怎么操作呢?

```
pd.DataFrame.from_dict(data,orient='index',columns = ['value1','value2'])
```

运行结果：

```
    value1  value2
a      1       2
b      2       3
```

如果进一步想让a、b生成列的话，调用reset_index方法即可。

```
pd.DataFrame.from_dict(data,orient='index',columns =
['value1','value2']).reset_index().rename(columns = {'index':'key'})
```

运行结果：

```
    key value1  value2
 0  a      1       2
1   b      2       3
```

但是如果我们想把字典的key和value分别生成两列，如何操作呢?
一种方法是：

```
pd.DataFrame(list(data.items()),columns = ['key','value'])
>>>
>>>     key value
>>> 0   a   [1, 2]
>>> 1   b   [2, 3]
```

还有一种方法依然是利用from_dict,不过就需要将value中的list提前转化成字符串，然后再进行操作即可。

1.3 np.array

numpy是比pandas更底层一些的数据操作工具，pandas的很多操作也是基于numpy进行的，比如numpy就支持直接读取txt文件。比如有这样一个txt文件：

```
%%%
1 10 0.45240003518120125 1.0000444454321133 0.10599999999999998 1.0599999999999998e-01 0.22999999999999998 0.472
2 20 0.43459179018909283 1.1133165687809157 0.07834109593771774 7.834109593717736e-02 0.2089183326689947 0.3863815370463022
3 30 0.40767309706715493 1.269342944674328 0.07190653014564094 7.190653014564094e-02 0.17795528298262073 0.4136993009059622
4 40 0.3859105442514819 1.3433376585083965 0.066153468987387 6.6153468987386999e-02 0.1477849202849159 0.261667203674047
```

一共有4行8列的数据，数据间用空格隔开，表头带有%，那么读取的时候可以用loadtxt函数进行导入：

```
data = np.loadtxt('fit.txt', delimiter=None, comments='%',  usecols=(0, 1, 4,5))
```

运行结果：

```
array([[ 1.        , 10.        ,  0.106     ,  0.106     ],
       [ 2.        , 20.        ,  0.0783411 ,  0.0783411 ],
       [ 3.        , 30.        ,  0.07190653,  0.07190653],
       [ 4.        , 40.        ,  0.06615347,  0.06615347]])
```

可以看到数据自动剔除了表头，并且只用了其中指定的列。接下来就可以将array导入到pandas中：

```
Data = pd.DataFrame(data, index = np.arange(len(data)), columns=['a','b','c','d'])
```

我们就可以得到类似用list构建DataFrame的效果了：

```
      a   b         c             d
0   1.0   10.0  0.106000    0.106000
1   2.0   20.0  0.078341    0.078341
2   3.0   30.0  0.071907    0.071907
3   4.0   40.0  0.066153    0.066153
```

1.4 其他方式

当然需要导入文本并不规则的时候，可以考虑直接利用python中的文件读取来一行一行的读取文件，然后利用json或者re等字符串处理包来处理数据，最后整合成DataFrame:

```
with open(path, "r") as load_f:
     l = f.readlines()
```

当然这个方法要结合具体的数据来看，这里就不展开介绍了。

2. 文本格式数据格式数据

2.1 CSV文件

2.1.1 导入CSV文件

常用参数解析：

```
pandas.read_csv(filepath_or_buffer, sep=',', header='infer', names=None,
indxe_col=None)
```

- filepath_or buffer: str, path object or file-like object。指定传入的文件路径，必须传入的参数。
- sep: str。指定分隔符，默认是逗号分隔符。
- header: int, list or int。指定行数用来作为列名。默认是如果没有传入names参数，则header=0，用第一行作为列名，否则header=None，以传入的names作为列名。另外如果传入的是list，例如[0,1,3]，则是以第1、2、4这些行作为多级列名，且中间的行，第3行会被忽略，数据从第5行开始。
- names: array-like, optional。指定文件的列名。如果文件中没有标题行，建议传入此参数。
- index_col: int, str, or sequence of int / str, or False。指定文件的索引，默认为None。

ex1.csv内容如下：

```
ID,name,age,city,message
A001, 小明,18, 北京,hello
A002, 小王,20, 杭州,world
A003, 小北,21, 上海,hello
A004, 张三,18, 北京,pandas
```

导入ex1.csv

```
df = pd.read_csv('examples/ex1.csv')
df
```

运行结果：

```
     ID     name    age   city   message
0   A001    小明      18    北京      hello
1   A002    小王      20    杭州      world
2   A003    小北      21    上海      hello
3   A004    张三      18    北京      pandas
```

ex2.csv文件没有标题行

```
A001|小明|18|北京|hello
A002|小王|20|杭州|world
A003|小北|21|上海|hello
A004|张三|18|北京|pandas
```

设置sep和header参数，导入ex2.csv

```
df2 = pd.read_csv('examples/ex2.csv',sep='|',header=None)
df2
```

运行结果：

```
    0       1     2     3     4
0   A001    小明    18    北京    hello
1   A002    小王    20    杭州    world
2   A003    小北    21    上海    hello
3   A004    张三    18    北京    pandas
```

设置sep和names参数，此时header默认为None

```
df3 = pd.read_csv('examples/ex2.csv',sep='|', names=
['ID','name','age','city','message
```

运行结果：

```
    ID     name   age    city    message
0   A001    小明      18    北京      hello
1   A002    小王      20    杭州      world
2   A003    小北      21    上海      hello
3   A004    张三      18    北京      pandas
```

对ex1.csv设置多级标题，将第1、2、4行作为标题，数据从第5行开始

```
df4 = pd.read_csv('examples/ex1.csv',header=[0,1,3])
df4
```

对ex1.csv设置多级标题，将第1、2、4行作为标题，数据从第5行开始

```
    ID    name  age   city  message
    A001  小明   18     北京    hello
    A003  小北   21     上海    hello
0   A004   张三   18    北京    pandas
```

导入ex1.csv，指定索引为message一列

```
df5 = pd.read_csv('examples/ex1.csv',index_col='ID')
df5
```

运行结果：

```
      name    age   city  message
ID
A001    小明    18    北京    hello
A002    小王    20    杭州    world
A003    小北    21    上海    hello
A004    张三    18    北京    pandas
```

导入ex1.csv，指定第1和2列作为多重索引

```
df6 = pd.read_csv('examples/ex1.csv',index_col=[0,1])
df6
```

运行结果：

```
            age   city    message
ID    name
A001    小明    18    北京    hello
A002    小王    20    杭州    world
A003    小北    21    上海    hello
A004    张三    18    北京    pandas
```

2.1.2 导出csv数据

参用参数解析：

```
DataFrame.to_csv(path_or_buf, index=True, header=True, sep=',', encoding='utf-8')
```

- path_or_buf: str or file handle。指定保存文件路径，必须传入的参数，默认为None。
- index: bool。导出的csv是否包含索引，默认为True。
- header: bool or list of str。导出的csv是否包含标题行，默认为True。
- sep: str。指定导出的csv文件的分隔符，默认为逗号分隔符。
- encoding: str。指定导出的csv文件的编码，默认为utf-8。

```
# 导出文件
df.to_csv("output/out_ex1.csv",index=False)
```

2.2 excel文件

2.2.1 导入excel文件

常用参数解析：

```
pd.read_excel(io, sheet_name=0, header=0, names=None, index_col=None)
```

read_excel和read_csv的用法差不多，一个需要注意的参数是sheet_name。这个参数是指定读取该excel中具体哪个表的数据，默认为0，即为第一个表。如果传入1，则为第2个表；可指定传入表名，如"Sheet1"；也可传入多个表，如[0,'Sheet3']，传入第一个表和名为'Sheet3'的表。
读取ex1.xlsx文件，默认为读取第一个表

```
df = pd.read_excel("examples/ex1.xlsx")
df
```

运行结果：

```
>>>
>>>     col_1   col_2   col_3   col_4   col_5
>>> 0     a       b       c       d       1
>>> 1     e       f       g       h       2
>>> 2     i       j       k       l       3
>>> 3     m       n       o       p       4
```

读取ex1.xlsx文件的第2个表

```
df2 = pd.read_excel("examples/ex1.xlsx",sheet_name=1)
df2
```

运行结果：

```
    col_1   col_2   col_3   col_4   col_5
0     aa      bb      cc      dd      11
1     ee      ff      gg      hh      22
2     ii      jj      kk      ll      33
3     mm      nn      oo      pp      44
```

读取ex1.xlsx文件的第2个表和名为"Sheet3"的表，返回的是对象是OrderedDict。OrderedDict是dict的子类，与dict不同的是，它记住了内容的顺序。

```
od = pd.read_excel("examples/ex1.xlsx",sheet_name=[1,'Sheet3'])
od
```

运行结果：

```
OrderedDict([(1,   col_1 col_2 col_3 col_4  col_5
                   0    aa    bb    cc    dd     11
                   1    ee    ff    gg    hh     22
                   2    ii    jj    kk    ll     33
                   3    mm    nn    oo    pp     44),
             ('Sheet3',   col_1 col_2 col_3 col_4  col_5
                   0   aaa   bbb   ccc   ddd    111
                   1   eee   fff   ggg   hhh    222
                   2   iii   jji   kkk   lll    333
                   3   mmm   jjj   ooo   ppp    444)])
```

在这个orderedDict中，有两个key。第一个key是1，对应的value为该表的内容；第二个key是'Sheet3',对应的value是Sheet3表格的内容。我们选取key，就能得到相应的value。

```
od[1]
```

运行结果：

```
    col_1   col_2   col_3   col_4   col_5
0     aa      bb      cc      dd      11
1     ee      ff      gg      hh      22
2     ii      jj      kk      ll      33
3     mm      nn      oo      pp
```

```
od['Sheet3']
```

运行结果：

```
    col_1   col_2   col_3   col_4   col_5
0     aaa     bbb     ccc     ddd     111
1     eee     fff     ggg     hhh     222
2     iii     jji     kkk     lll     333
3     mmm     jjj     ooo     ppp     444
```

2.2.2 导出excel文件

常用参数解析：

```
DataFrame.to_excel(excel_writer, sheet_name='Sheet1',index=True)
```

- excel_writer: str。指定保存文件路径。
- sheet_name: str。指定excel文件的表名，默认为'Sheet1'。
- index：bool。是否保存索引，默认为True。

```
df.to_excel('output/out_ex1.xlsx')
df.to_excel('output/out_ex2.xlsx',sheet_name='结果',index=False)
```

2.3 txt文件

2.3.1 导入txt文件

常用参数解析：

```
pandas.read_table(filepath_or_buffer, sep='\t', header='infer', names=None, index_col=None)
```

read_table与read_csv的唯一区别是，read_csv默认的sep参数是逗号分隔符，而read_table默认是'\t'，制表符。所以这两个方法是通用的，只要设置好分隔符，都可以读取csv和txt文件。
ex3.txt文件的内容如下：

```
ID      name    age      city
A001      小明    18  北京
A002      小王    20  杭州
A003      小北    21  上海
A004      张三    18  北京
A005      李四    23  上海
A006      小思    24  广州
A007      王五    24  上海
A008      小哇    19  北京
A009      黎明    25  上海
A010      夕阳       23  杭州
```

导入ex3.txt文件

```
df = pd.read_table('examples/ex3.txt')
df
```

运行结果：

```
     ID     name  age   city
0   A001    小明    18    北京
1   A002    小王    20    杭州
2   A003    小北    21    上海
3   A004    张三    18    北京
4   A005    李四    23    上海
5   A006    小思    24    广州
6   A007    王五    24    上海
7   A008    小哇    19    北京
8   A009    黎明    25    上海
9   A010    夕阳    23    杭州
```

将sep参数设置为逗号，同样能读取ex1.csv文件

```
df2 = pd.read_table('examples/ex1.csv',sep=',')
df2
```

运行结果：

```
    ID    name    age   city    message
0   A001    小明    18    北京      hello
1   A002    小王    20    杭州      world
2   A003    小北    21    上海      hello
3   A004    张三    18    北京      pandas
```

2.3.2 导入txt文件

使用to_csv的方法

```
df2.to_csv('output/ex3.txt',sep='\t')
```

2.4 csv和xlsx的选择

当我们可以选择保存为csv或者xlsx格式，方便下次可以使用的时候，是选择保存为csv还是excel呢？除了考虑csv和excel文件大小之外（相同的数据下excel文件比csv文件小），这里可以考虑下read_csv和read_xlsx的性能问题。在stackoverflow上有人对这两种导入方法进行了一个简单的测试。

测试文件：同样的数据集（分别是320MB的csv文件和16MB的xlsx文件）

电脑硬件：i7-7700k，SSD

python环境：Anaconda Python 3.5.3, pandas 0.19.2

	用时
pd.read_csv('foo,csv')	2s
pd.read_excel('foo.xlsx')	15.3s
df.to_csv('bar.csv',index=False)	10.5s
df.to_excel('bar.xlsx',index=False)	34.5s

2.5 json

2.5.1 导入json文件

常用参数解析：

```
pandas.read_json(path_or_buf=None, orient=None, typ='frame')
```

- path_or_buf: 指定文件路径，默认为None，必须传入的参数。
- orient: json字符串格式，默认为None。这里有split,records,index,columns,values五种选择可选。
- typ: 要转换为series还是dataframe，默认为frame。当typ=frame时，orient可选split/records/index,默认

为columns；当typ=series,orient可选split/records/index/columns/value,orient默认为index。

split格式： dict like {index -> [index], columns -> [columns], data -> [values]}, 例如下面的ex4.json文件。

```
{"index":[1,2,3,4],
     "columns":["ID","age","city","name"],
     "data":[["A001",18,"北京","小明"],
             ["A002",20,"杭州","小王"],
             ["A003",21,"上海","小北"],
             ["A004",18,"北京","张三"]]
```

导入ex4.json

```
df = pd.read_json('examples/ex4.json',orient="split")
df
```

运行结果：

```
    ID    age     city      name
1   A001    18    北京        小明
2   A002    20    杭州        小王
3   A003    21    上海        小北
4   A004    18    北京        张三
```

records格式：list like [{column -> value}, ..., {column -> value}]，例如下面的ex5.json文件。

```
[{"ID":"A001","name":"小明","age":18,"city":"北京"},
{"ID":"A002","name":"小王","age":20,"city":"杭州"},
{"ID":"A003","name":"小北","age":21,"city":"上海"},
{"ID":"A004","name":"张三","age":18,"city":"北京"}]
```

导入ex5.json

```
df1 = pd.read_json('examples/ex5.json',orient="records")
df1
```

运行结果同上。
如果是转为series格式：

```
pd.read_json('examples/ex5.json',orient="records",typ="series")
```

运行结果：

```
0    {'ID': 'A001', 'name': '小明', 'age': 18, 'city'...
1    {'ID': 'A002', 'name': '小王', 'age': 20, 'city'...
2    {'ID': 'A003', 'name': '小北', 'age': 21, 'city'...
3    {'ID': 'A004', 'name': '张三', 'age': 18, 'city'...
dtype: object
```

index格式: dict like {index -> {column -> vlaue}}，例如下面的ex6.json文件。

```
{"1": {"ID":"A001","name":"小明","age":18,"city":"北京"},
"2": {"ID":"A002","name":"小王","age":20,"city":"杭州"},
"3":{"ID":"A003","name":"小北","age":21,"city":"上海"},
"4":{"ID":"A004","name":"张三","age":18,"city":"北京"},
}
```

导入ex6.json

```
df2 = pd.read_json('examples/ex6.json',orient="index")
df2
```

运行结果：

```
    ID    age     city    name
1   A001    18    北京      小明
2   A002    20    杭州      小王
3   A003    21    上海      小北
4   A004    18    北京      张三
```

如果是转为series格式：

```
pd.read_json('examples/ex6.json',orient="index",typ="series")
```

运行结果：

```
0    {'ID': 'A001', 'name': '小明', 'age': 18, 'city'...
1    {'ID': 'A002', 'name': '小王', 'age': 20, 'city'...
2    {'ID': 'A003', 'name': '小北', 'age': 21, 'city'...
3    {'ID': 'A004', 'name': '张三', 'age': 18, 'city'...
dtype: object
```

columns格式： dict like {column -> {index -> value}}，例如下面的ex7.json文件。当typ='frame'时，orient默认为这个格式。

```
{
    "ID":{"1":"A001","2":"A002","3":"A003","4":"A004"},
    "name":{"1":"小明","2":"小王","3":"小北","4":"张三"},
    "age":{"1":18,"2":20,"3":21,"4":18},
    "city":{"1":"北京","2":"杭州","3":"上海","4":"北京"},
}
```

导入ex7.json

```
df3 = pd.read_json('examples/ex7.json',orient="columns")
# 或者
df3 = pd.read_json('examples/ex7.json')
```

运行结果：

```
    ID      name  age   city
1   A001    小明    18    北京
2   A002    小王    20    杭州
3   A003    小北    21    上海
4   A004    张三    18    北京
```

用columns格式读取ex6.json，其实与index格式的结果是行列的转置。

```
df4 = pd.read_json('examples/ex6.json',orient="columns")
df4
```

运行结果：

```
       1       2       3       4
ID   A001   A002    A003    A004
ge   18     20      21      18
city 北京 杭州      上海      北京
name 小明 小王      小北      张三
```

values格式： just the values array，例如下面的ex8.json文件。

```
[["A001","小明",18,"北京"],
["A002","小王",20,"杭州"],
["A003","小北",21,"上海"],
["A004","张三",18,"北京"]]
```

导入ex8.json

```
df5 = pd.read_json('examples/ex8.json',orient="values")
df5
```

运行结果：

```
    0       1     2     3
0   A001    小明    18    北京
1   A002    小王    20    杭州
2   A003    小北    21    上海
3   A004    张三    18    北京
```

2.5.2 导出json文件

常用参数解析：

```
DataFrame.to_json(path_or_buf=None, orient=None,index=True)
```

- orient: string。指定导出json的格式。DataFrame默认是columns，Series默认是index

dataframe导出json，命名为out_ex4.json

```
df.to_json("output/out_ex4.json")
```

series导出json，命名为out_ex5.json

```
se = pd.read_json('examples/ex6.json',orient="index",typ="series")
se.to_json("output/out_ex5.json")
```

3 数据库

3.1 MySQL

在开始之前，请确保环境中的python为3.x版本，且已经安装并开启mysql服务。这里我们使用pymysql库来连接mysql。首先需要通过pip安装pymysql。安装后，可以通过import语句检验是否已经安装成功。如果没有报错，则说明安装成功。

```
pip install pymysql
```

```
import pymysql
# 打开数据库连接
# 注意在进行这一步之前要先创建好数据库。如果数据库不存在，这一步会报错。
conn =
pymysql.connect(host="localhost",user="username",password="password",db="dbtest")
# 创建一个游标对象
cursor = conn.cursor()
```

3.1.1 数据导入mysql

```
# 创建数据库表
sql_createTb = """CREATE TABLE user (
                  ID CHAR(4) NOT NULL,
                  name CHAR(20),
                  age INT,
                  city CHAR(20)
                  )
                  """
# 执行SQL语句
cursor.execute(sql_createTb)

# 插入数据
insert1 = "INSERT INTO user(ID,name,age,city) values('A001','小明',18,'北京')"
insert2 = "INSERT INTO user(ID,name,age,city) values('A002','小王',20, '杭州');"
insert3 = "INSERT INTO user(ID,name,age,city) values('A003','张三',18, '北京');"
insert4 = "INSERT INTO user(ID,name,age,city) values('A004','张三',18, '北京');"
insert5 = "INSERT INTO user(ID,name,age,city) values('A005','李四',23, '上海');"
insert6=  "INSERT INTO user(ID,name,age,city) values('A006','小思',24, '广州');"
# 执行SQL语句
for sql_insert in [insert1,insert2,insert3,insert4,insert5,insert6]:
    cursor.execute(sql_insert)
# pymysql默认是没有开始自动提交事务的
# 所以在对更新数据库的时候，一定要手动提交事务
conn.commit()

# 更新数据
sql_update = "update user set city='深圳' where ID='A001'"
# 执行SQL语句
cursor.execute(sql_update)
# 提交事务
conn.commit()

# 删除数据
sql_delete = "delete from user where ID='A004'"
# 执行SQL语句
cursor.execute(sql_delete)
# 提交事务
conn.commit()
```

3.1.2 读取mysql数据

通过sql语句查询数据

```
# 查询数据
sql_search  = "SELECT * FROM user"
cursor.execute(sql_search)
# 查看结果
results = cursor.fetchall()
results
```

运行结果：

```
 (('A001', '小明', 18, '深圳'),
  ('A002', '小王', 20, '杭州'),
  ('A003', '张三', 18, '北京'),
  ('A005', '李四', 23, '上海'),
  ('A006', '小思', 24, '广州')
```

将结果转换为dataframe格式

```
df = pd.DataFrame(list(results))
df.columns = ['ID','name','age','city']
df
```

运行结果：

```
    ID      name    age   city
0   A001    小明      18    深圳
1   A002    小王      20    杭州
2   A003    张三      18    北京
3   A005    李四      23    上海
4   A006    小思      24    广州
```

关闭数据库连接：

```
cursor.close()
conn.close()
```

3.2 PostgreSQL

psycopg2是Python语言的PostgreSQL数据库接口之一，这里我们使用psycopg2连接，首先同样请确保环境中已经安装postgreSQL，以及已通过pip安装psycopg2了。

3.2.1 读取postgreSQL的数据

```
import psycopg2

# 连接数据库
```

```
conn = psycopg2.connect(database = 'name', user = 'admin', password = '123456',
host = '10.10.10.10', port = '5432')

curs=conn.cursor()

# 编写Sql，只取前两行数据
sql = 'select * from table_name limit 2'

#  数据库中执行sql命令
curs.execute(sql)

#获得数据
data = curs.fetchall()
```

返回的data结果是一个以各行数据为元组的列表，如下：

```
[('L002','WKQ1','WZ1A','WZ1A','L','WZ01-12',\
 '10073864791','R5400','5','18','36','362.29',\
'372.57','351','20190311','20190317','11','0','0',\
'0,8','3','3','0.83','3','3','20190310'),
('L002','WKQ1','WZ1A','WZ1A','L','WZ01-14',\
 '10073864791','R5400','5','18','36','300.29',\
'372.57','351','20190311','20190317','11','0','0',\
'0,8','3','3','0.83','3','3','20190310')]
```

可以通过pandas对data进行进一步处理：pd.DataFrame(data)。
在insert的时候，需要注意以下几点：

1. 表中的字段不需要加引号；
2. 插入的每行数值用括号包围，其中各个字段以逗号间隔，字符串型必须加引号；
3. 以上sql命令可见，一条sql命令可以插入多条数据，只需要连接各个数据，最终commit一次就好；
4. 另外在写入PG的时候，应该注意PG中的数据如果出现单引号“'”会出现错误，所以必须先使用replace替换成其他的内容方可写入。

3.2.2 数据写入postgreSQL

```
#编辑写入数据的sql
insert_sql = \
"insert into table_name
(warehouse_code,storehouse_code,zone_code,picking_zone_code,picking_type,location_code,\
gds_id,kunnr,yest_tot_qty,week_tot_qty,month_tot_qty,week_avg_prlab,month_avg_prlab,\
present_storage,start_date,end_date,week_flag,unsaleble_flag,super_A,sales_segment,\
present_class,week_avg_qty,month_avg_qty,now_level,pred_level,statis_date,\
prediction_class,action,target_area)
values
```

```
 ('L121','6','IWDX','IWDX','L','IWDX-001-
0101','120339999','S70227820','0','0','2','1','1','1','20190311','20190317','11','0
','0','0,7','1','0','0','1','1','20190310','2','2','targ_null'),\
 ('L002','WKQ1','WZ1A','WZ1A','L','WZ0114','10073864791','R5400','5','18','36','300
.29',\
'372.57','351','20190311','20190317','11','0','0',\
'0,8','3','3','0.83','3','3','20190310')"

curs.execute(insert_sql)

#提交数据
conn.commit()
#关闭指针和数据库
curs.close()
conn.close()
```

3.3 支持多种数据库 — SQLAlchemy

SQLAlchemy是python下的一款数据库对象关系映射工具（ORM工具），能满足大多数数据库操作需求，且支持多种数据库引擎，能连接上文提及的MySQL, PostgreSQL, Oracle之外，还支持Mircosoft SQL Server, SQLite等的数据库。另外在pandas中，配合使用SQLalchemy连接数据库，可以实现更简便高效的查询和导入数据的操作，因为pandas已经帮你写好一些常用的方法了。

下面我们以连接mysql数据库为例子介绍用法，首先还是需要先通过pip安装sqlalchemy和pymysql。

```
from sqlalchemy import create_engine
# 连接数据库
# 数据库名字
db = 'dbtest'
# username为用户名，password为密码
engine = create_engine("mysql+pymysql://username:password@localhost:3306/%s?
charset=utf8mb4" % db, echo=False)
# 如果需要连接其他数据库，需要更改这里的create_engine
# 如postgresql,
create_engine('postgresql+psycopg2://scott:tiger@localhost/mydatabase')
# 具体可以查看 https://docs.sqlalchemy.org/en/13/core/engines.html
```

3.3.1 读取数据库数据

在pandas中，我们可以通过read_sql_table和read_sql的方法来读取数据库，pandas会帮我们将结果直接转为dataframe的格式，这对于需要dataframe格式数据的来说是非常方便的。

```
# 读取某个表的数据
pandas.read_sql_table(table_name, con)
# 查询sql
pandas.read_sql(sql, con)
```

读取数据下现有所有的表

```
pd.read_sql('show tables', engine)
```

运行结果：

```
    Tables_in_dbtest
0   user
```

读取数据库下某个表的数据

```
pd.read_sql_table(user, engine)
```

运行结果：

```
    ID      name    age   city
0   A001    小明      18    深圳
1   A002    小王      20    杭州
2   A003    张三      18    北京
3   A005    李四      23    上海
4   A006    小思      24    广州
```

根据sql语句查询数据：

```
# 查询sql
sql = "SELECT * FROM user WHERE age=18"
pd.read_sql(sql, engine)
```

运行结果：

```
    ID      name age    city
0   A001    小明    18    深圳
1   A003    张三    18    北京
```

3.3.2 数据导入数据库

将dataframe格式的数据导入到数据库中，我们可以使用to_sql的方法。

```
DataFrame.to_sql(name, con, if_exists='fail')
```

- name：导入库的表的名字
- if_exists：默认为"fail"，表示如果表不存在，直接报错；可选"replace"，导入的dataframe直接覆盖该表；可选"append"，将数据添加到表的后面。

```
df = pd.read_csv('examples/sql.csv')
# 将df导入数据库中
table_name = 'user2'
df.to_sql(table_name, engine, if_exists='append', index=False)
```

3.4 MongoDB

MongoDB 是目前最流行的 NoSQL 数据库之一，使用的数据类型 BSON（类似 JSON）。这里我们使用PyMongo连接MongoDB数据库。

```
# 导入前需要pip安装pymongo，并开始mongoDB服务
from pymongo import MongoClient
# 连接mongoDB数据库
myclient = MongoClient('mongodb://localhost:27017/')
```

3.4.1 将数据导入数据库

```
# 创建一个集合
mydb = myclient["dbtest"]
mycol = mydb["user"]
# 导入一条数据,data的格式为{col:value}
data_one = {"ID":"A011","name":"小黑","age":18,"city":"深圳"}
mycol.insert_one(data_one)
# 导入多条数据
data_many = [{"ID":"A012","name":"小红","age":23,"city":"深圳"},
{"ID":"A013","name":"小白","age":30,"city":"深圳"},
{"ID":"A014","name":"小蓝","age":24,"city":"深圳"}]
mycol.insert_many(data_many)
# 将dataframe转为json后导入mongo
import json
def df2mongo(df, mycol):
    records = json.loads(df.T.to_json()).values()
    result = mycol.insert_many(records)
    return result
df = pd.read_table('examples/sql.csv')
df2mongo(df,mycol)
```

3.4.2 读取数据库数据

```
# 读取mongo某个集合的所有数据
mycol.find()
# 读取mongo某个集合的所有数据，并转为dataframe数据格式
df = pd.DataFrame(list(mycol.find()))
```

```
# 指定条件查询,返回所有符合条件的数据
myquery = { "city": "上海" }
mydoc = mycol.find(myquery)
for x in mydoc:
    print(x)
```

运行结果：

```
[{'_id': ObjectId('5d51625fae73ac0c50b1277d'),
  'ID': 'A003',
  'name': '小北',
  'age': 21,
  'city': '上海'},
 {'_id': ObjectId('5d51625fae73ac0c50b1277f'),
  'ID': 'A005',
  'name': '李四',
  'age': 23,
  'city': '上海'},
 {'_id': ObjectId('5d51625fae73ac0c50b12781'),
  'ID': 'A007',
  'name': '王五',
  'age': 24,
  'city': '上海'},
 {'_id': ObjectId('5d51625fae73ac0c50b12783'),
  'ID': 'A009',
  'name': '黎明',
  'age': 25,
  'city': '上海'}]
```

```
# 指定条件查询,返回符合条件的指定条件数据
mydoc = mycol.find(myquery).limit(1)
for x in mydoc:
    print(x)
```

运行结果：

```
{'_id': ObjectId('5d51625fae73ac0c50b1277d'), 'ID': 'A003', 'name': '小北', 'age': 21, 'city': '上海'}
```

```
# 高级查询，例返回所有年龄超过24岁的用户
myquery = { "age": { "$gt": 24 } }
mydoc = mycol.find(myquery)
for x in mydoc:
    print(x)
```

运行结果：

```
{'_id': ObjectId('5d51625fae73ac0c50b12779'), 'ID': 'A013', 'name': '小白', 'age': 30, 'city': '深圳'}
{'_id': ObjectId('5d51625fae73ac0c50b12783'), 'ID': 'A009', 'name': '黎明', 'age': 25, 'city': '上海'}
```

参考文献：

1. 《Python for Data Analysis》
2. pandas官方文档：https://pandas.pydata.org/pandas-docs/stable/index.html
3. https://stackoverflow.com/questions/31362573/performance-difference-in-pandas-read-table-vs-read-csv-vs-from-csv-vs-read-e
4. https://www.runoob.com/python3/python3-mysql.html
5. https://www.runoob.com/python3/python-mongodb.html

作者：杨士锦，周岩，书生

责编：杨士锦，邢昱

审稿责编：书生，周岩